

Module 1: Getting started with HTML & CSS

Welcome to the exciting world of web development! This week, you've laid the foundation with HTML for structure and CSS for style. Let's recap the awesomeness!

HTML - The Skeleton of Your Webpage

Imagine building a house. HTML is the blueprint and the structural frame!

- **What is HTML?**
 -
 - **HyperText Markup Language.** It's not a programming language, but a *markup* language that tells browsers how to display content. Think of it as giving instructions!
- **Tags: Your HTML Building Blocks**
 - Most tags come in pairs: an **opening tag** (`<tag>`) and a **closing tag** (`</tag>`). Everything in between is the content.
 - **Example:** `<p>This is a paragraph.</p>`
 - Some tags are **self-closing** (or void tags) like `<br>` (for a line break) or `<hr>` (for a horizontal rule). They don't need a closing tag because they don't enclose content.

HTML Page Structure: The Basic Blueprint

```
HTML
<!DOCTYPE html> <html lang="en"> <head>
  <meta charset="UTF-8"> <meta name="viewport" content="width=device-width,
initial-scale=1.0"> <title>My Awesome Page</title> </head>
<body>
  </body>
</html>
```

- - The `<head>` contains meta-information about the page (not visible).
 - The `<body>` holds all the content you actually see.
- **Creating an HTML File:**
 - Simply save your text file with a `.html` or `.htm` extension (e.g., `index.html`). Open it with any web browser!
- **Basic HTML Tags: Your First Words on the Web!**
 - `<b>` & `<strong>`: Make text **bold**. `<strong>` has semantic importance (it *emphasizes* the text).
 - `<br>`: A line break. No closing tag!
 - `<hr>`: A horizontal rule (a line). No closing tag!

- `<p>`: For paragraphs of text.
- `<i>` & `<em>`: Make text *italic*. `<em>` has semantic importance (it *emphasizes* the text).
- `<center>`: (Outdated!) Centers content. Use CSS for this now!
- `<u>`: (Outdated!) Underlines text. Use CSS for this now!
- `<h1>` to `<h6>`: Heading tags. `<h1>` is the most important, `<h6>` the least.
- ` `: Non-breaking space. Gives you an extra space where you need it!

Essential Tags - Bringing Your Content to Life

Now that you know the basics, let's add some more cool stuff!

- **Image Tag (`<img>`) & Attributes:**

- `<img>` is a self-closing tag. It's how you put pictures on your page!
- **src (source)**: The *most important* attribute! It tells the browser where to find the image file. Can be a local path or a URL.
 - **Example**: ``
- **alt (alternative text)**: Crucial for accessibility! Describes the image for screen readers and if the image fails to load.
 - **Example**: ``
- **title**: Text that appears when you hover over the image.
 - **Example**: ``
- **width & height**: Set the dimensions of the image in pixels (px).
 - **Example**: ``

- **Anchor Tag (`<a>`) & Attributes: The Link to Everywhere!**

- This is how you create hyperlinks to other pages or sections within the same page.
- **href (hypertext reference)**: The destination URL.
 - **Example**: `<a href="https://www.google.com">Go to Google</a>`
 - **Linking to another page in your project**: `<a href="about.html">About Us</a>`
- **target="_blank"**: Opens the linked page in a *new tab*.
 - **Example**: `<a href="https://www.google.com" target="_blank">Google in a new tab</a>`

- **HTML Lists: Organizing Your Info!**

Ordered Lists (): Numbered lists. Each item is an (list item).

HTML

```
<ol>
  <li>First item</li>
  <li>Second item</li>
</ol>
```

Unordered Lists (): Bulleted lists. Each item is an .

HTML

```
<ul>
  <li>Bullet point one</li>
  <li>Bullet point two</li>
</ul>
```

-
- **Create Home Page Using All Basic Tags:**
 - This was your first mini-project! You combined headings, paragraphs, images, links, and lists to build a structured webpage. Great job!

Introduction to CSS - Making it Pretty!

HTML provides the content, but CSS makes it *look good*!

- **What is CSS?**

Cascading Style Sheets. It describes how HTML elements should be displayed on screen. Think of it as the interior designer for your house.

- **CSS Selectors:** How you pick which HTML element to style.

Element Selector: Styles *all* instances of a specific HTML element.

CSS

```
p {
  color: blue; /* All paragraphs will be blue */
}
```

Class Selector: Styles elements with a specific `class` attribute. You can reuse classes!

HTML

```
<p class="highlight">Important text</p>
```

```
<h2 class="highlight">Important Heading</h2>
```

CSS

```
.highlight {  
  background-color: yellow;  
}
```

ID Selector: Styles a *unique* element with a specific `id` attribute. An `id` should only be used once per page!

HTML

```
<div id="main-header">Welcome!</div>
```

CSS

```
#main-header {  
  font-size: 3em;  
}
```

- **Internal, External, and Inline CSS: Where to Put Your Styles?**

Inline CSS: Styles applied directly to an HTML element using the `style` attribute.

- **Pros:** Quick for one-off styles.
- **Cons:** Messy, hard to maintain, no separation of concerns. **Avoid for larger projects!**
- **Example:** `<p style="color: red; font-size: 16px;">This text is red.</p>`

Internal CSS: Styles defined within a `<style>` tag in the `<head>` section of your HTML.

- **Pros:** Good for single-page styles, keeps styles with the HTML.
- **Cons:** Not scalable for multiple pages.

Example:

```
HTML
<head>
  <style>
    h1 {
      color: green;
    }
  </style>
</head>
```

External CSS: Styles defined in a separate `.css` file and linked to your HTML. **This is the best practice!**

- **Pros:** Clean separation, reusable across multiple pages, easy to maintain.

Example (in HTML `<head>`):

```
HTML
<link rel="stylesheet" href="style.css">
```

- (Then, all your CSS rules go in `style.css`)

- **CSS Properties: Your Styling Toolbox!**

color: Sets the text color. (e.g., `color: blue;` `color: #FF0000;` `color: rgb(255, 0, 0);`)

background-color: Sets the background color of an element.

- **Example:** `background-color: lightgray;`

font-weight: Controls the boldness of text. (e.g., `font-weight: bold;` `font-weight: 700;` `font-weight: normal;`)

Layout Distribution with `<div>` - Organizing Your Content!

Now that we can style individual elements, how do we group them and arrange them on the page? Enter the `<div>`!

- **What is a `<div>` tag?**
 - The `<div>` (division) tag is a generic container for flow content. It has no semantic meaning on its own, but it's incredibly useful for grouping elements

- together so you can apply styles or scripts to them. Think of it as an empty box you can label and then put other things inside.
- **How it Works:**
 - `<div>` elements are **block-level** by default. This means they take up the full available width and start on a new line. They essentially create a "block" of content.
 - They are the workhorses for creating sections, columns, and overall page layouts.
- **`<div>` Calculation and Flow Understanding:**
 - Since `divs` are block-level, they stack vertically by default.
 - Understanding the **Document Flow** is key: HTML elements are rendered in the order they appear in the source code. Block elements break the flow and start on a new line.
- **CSS Properties for `<div>` (and layout!):**
 - **width & height:** Set the exact dimensions of your `div`.

Example:

```
CSS
.my-box {
  width: 200px;
  height: 100px;
  background-color: skyblue;
}
```

- **flexbox (or `display: flex;`):** This is a *game-changer* for layout! Flexbox allows you to arrange items in a one-dimensional row or column. It makes aligning, distributing space, and ordering content super easy.
 - To use flexbox, you apply `display: flex;` to the *parent* container (the `div` that holds the items you want to arrange).

Example:

HTML

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
```

CSS

```
.container {
  display: flex; /* Makes the items inside flex items */
  justify-content: space-around; /* Distributes space between items */
  align-items: center; /* Vertically centers items */
}
.item {
  width: 50px;
  height: 50px;
  background-color: coral;
  margin: 5px;
}
```

- **Key Flexbox properties you'll explore later:** `flex-direction`, `justify-content`, `align-items`, `flex-wrap`, etc.

Essential CSS Properties - Fine-Tuning Your Design! ✨

Now let's get down to the nitty-gritty of making your text and elements look just right.

- **Basic CSS Properties (Text & More!):**

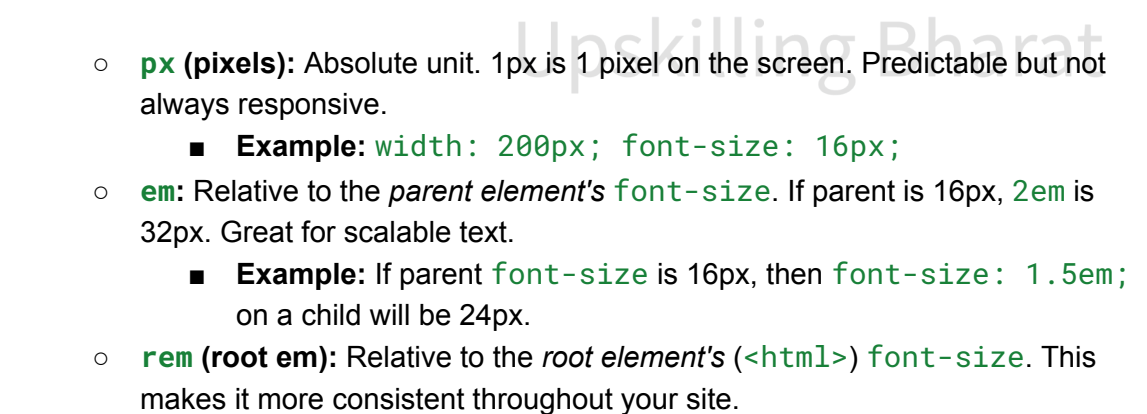
- `font-size`: Controls the size of the text. (e.g., `16px`, `1.2em`, `2rem`, `120%`)
- `font-weight`: Sets the thickness of characters. (e.g., `normal`, `bold`, `lighter`, `bolder`, `100` to `900`)
- `font-style`: Applies italic, oblique, or normal style. (e.g., `italic`, `normal`)
- `font-family`: Specifies the typeface (font). Always provide fallback fonts!
 - **Example:** `font-family: "Arial", sans-serif;`
- `text-align`: Aligns text within its container. (e.g., `left`, `right`, `center`, `justify`)
- `text-decoration`: Adds lines to text. (e.g., `none` (to remove underline from links), `underline`, `overline`, `line-through`)
- `text-transform`: Changes letter casing. (e.g., `uppercase`, `lowercase`, `capitalize`)

- **line-height**: Sets the height of a line of text. Affects spacing between lines. (e.g., 1.5 or 24px)
- **word-spacing**: Adjusts spacing between words.
- **letter-spacing**: Adjusts spacing between characters.
- **text-indent**: Indents the first line of text in a block.
- **list-style**: Controls the marker for list items (`<ul>` and `<ol>`). (e.g., none, disc, circle, square, decimal)
- **text-emphasis**: Adds emphasis marks to text. (Less common, but useful for specific languages).
- **line-break & word-break**: Controls how lines and words break within an element (advanced text control).
- **Border Properties: Adding Frames!**
 - Borders are lines that go around an element. They have three parts: width, style, and color.
 - **Shorthand Property**: `border: 2px solid red;` (width, style, color)
 - **Individual Directions**:
 - **border-left**: Styles the left border.
 - **border-top**: Styles the top border.
 - **border-right**: Styles the right border.
 - **border-bottom**: Styles the bottom border.
 - **Example**: `border-bottom: 1px dashed blue;`
 - **Common border-style values**: solid, dashed, dotted, double, none

CSS Box Model - The Foundation of Layout

This is fundamental! Every HTML element is treated as a rectangular box by the browser. Understanding this "box model" is crucial for positioning and spacing.

- **The Box Model: Content, Padding, Border, Margin** Imagine a picture frame.
 - **Content**: The actual image itself (your text, image, etc.).
 - **Padding**: The space *between* the content and the border. It's like the mat around the picture. **Padding is *inside* the border.**
 - **Border**: The line or frame around the padding.
 - **Margin**: The space *outside* the border, separating this box from other boxes. It's like the empty wall space around the framed picture.



- **Example:** If `html { font-size: 16px; }`, then `font-size: 1.5rem;` anywhere on the page will be 24px.
- **% (percentage):** Relative to the *parent element's* size. Useful for fluid layouts.
 - **Example:** `width: 50%;` means 50% of the parent's width.
- **box-sizing:** A CSS property that changes how the width and height of an element are calculated.
 - **content-box (default):** `width` and `height` only refer to the content area. Padding and border *add* to the total dimensions.
 - Total Width = `width` + `padding` + `border` + `margin`
 - **border-box:** `width` and `height` *include* padding and border. This is generally preferred as it makes layout calculations much easier and more intuitive.
 - Total Width = `width` (which now includes padding and border) + `margin`

Best Practice: Many developers set `box-sizing: border-box;` globally.

```
CSS
html {
  box-sizing: border-box;
}
*, *:before, *:after {
  box-sizing: inherit;
}
```